The Bizarre Geometry of High-Dimensional Space

Human intuition evolved to throw rocks and swing from branches in 3D space. Because of this, our mental models of geometry are fundamentally hardwired for 2 or 3 dimensions.
When we try to apply this intuition to high-dimensional Euclidean space (R^d), things break down catastrophically. Concepts like "volume," "distance," and "angles" behave in ways that feel entirely paradoxical.
In this notebook, we'll explore three classic, counterintuitive phenomena of high-dimensional space using simple Python simulations.

```
import numpy as np
import matplotlib.pyplot as plt

# Set a nice style for our plots
plt.style.use('seaborn-v0_8-darkgrid')
```

1. The Hollow Ball (Concentration of Measure)

Let's start with a simple question: **Where is the "inside" of a sphere?**
If you have a 3D watermelon, the tasty red part (the interior) makes up most of the fruit, while the green rind (the outer edge) is a relatively thin layer.
In high-dimensional space, **the watermelon is entirely rind.** If we take a d-dimensional ball of radius R=1, the volume scales as R^d. If we look at the volume of a slightly smaller inner ball of radius r=0.99, its relative volume is $(0.99)^d$. As d approaches infinity, $(0.99)^d$ approaches 0.
Let's simulate this by scattering random points uniformly *inside* a unit ball across different dimensions, and plotting the histogram of their distances from the center.

```
def sample_unit_ball(N, d):
    # To sample uniformly inside a d-dimensional ball, we generate Gaussian noise,
    # normalize it to get a random direction, and then scale by a uniform variable ^ (1/d)
    directions = np.random.normal(0, 1, (N, d))
    directions /= np.linalg.norm(directions, axis=1, keepdims=True)
    radii = np.random.uniform(0, 1, N) ** (1.0 / d)
    return directions * radii[:, np.newaxis]
```

```
N = 50000
dimensions = [2, 10, 50, 500]

fig, axes = plt.subplots(1, 4, figsize=(18, 4), sharey=True)

for ax, d in zip(axes, dimensions):
    points = sample_unit_ball(N, d)
    distances = np.linalg.norm(points, axis=1)

    ax.hist(distances, bins=50, color='coral', edgecolor='white', density=True)
    ax.set_title(f'Dimension (d) = {d}')
    ax.set_xlim(0, 1)
    ax.set_xlabel("Distance from center")

axes[0].set_ylabel("Density of Points")
plt.suptitle("Where is the mass of a Unit Ball?", fontsize=16, y=1.05)
plt.show()
```

Notice what happens!

- In **2D** (a circle), the mass increases linearly as you move outward.
- By **10D**, almost all the volume is pushed toward the outer edge.
- By **500D**, any random point you pick inside the ball is virtually guaranteed to be sitting precisely on the outer boundary. The interior is effectively empty.

2. The Spiky Hypercube (The Kissing Spheres Paradox)

Let's build a box. Imagine a d-dimensional hypercube centered at the origin, with side length 4. Its boundaries are at [-2, 2] in every direction.
Next, place a sphere of radius 1 in every single corner of this cube. Finally, place one "central" sphere at the origin, and expand it until it perfectly kisses all the corner spheres.

- In **2D** (a square), the central circle is small and safely tucked inside the square.
- In **3D**, the central sphere is a bit larger, but still entirely inside the box.

The distance to the corner spheres is the square root of d. Since the corner spheres have radius 1, the radius of the central sphere is r = √d - 1. But the distance from the origin to the walls of the bounding box is exactly 2. Let's plot the radius of this central sphere as dimensions increase.

```
d_vals = np.arange(1, 21)
central_radius = np.sqrt(d_vals) - 1

plt.figure(figsize=(10, 6))
plt.plot(d_vals, central_radius, marker='o', linewidth=2, color='indigo', label='Radius of Central Sphere')
plt.axhline(y=2, color='red', linestyle='--', linewidth=2, label='Boundary of the Bounding Box (r=2)')

# Highlight the crossover point
plt.fill_between(d_vals, 2, central_radius, where=(central_radius > 2), color='red', alpha=0.2, label='Sphere breaks out of the box')

plt.title("The Kissing Spheres Paradox", fontsize=16)
plt.xlabel("Number of Dimensions (d)", fontsize=12)
plt.ylabel("Radius of Central Sphere", fontsize=12)
plt.xticks(d_vals)
plt.legend(fontsize=12)
plt.show()
```

The Paradox: At d=9, the central sphere perfectly touches the walls of the box. For d > 9, **the central sphere bursts through the walls of the hypercube.** Even though it is pinned in place by corner spheres that are entirely *inside* the box, the central sphere bulges out into the space outside the box. In very high dimensions, a hypercube looks more like a spiky sea urchin than a solid block!

3. The Lonely Vectors (Almost Orthogonal)

If I ask you to visualize two random arrows (vectors) floating in 3D space, they are probably pointing in entirely different, random directions. The angle between them could be anything.
In high-dimensional space, the concept of "random directions" collapses. If you pick two random vectors in high dimensions, **they are almost guaranteed to be exactly perpendicular (90°) to each other.** Let's generate 1,000 random vectors in different dimensions, calculate the angle between every possible pair of them, and see what the distribution of angles looks like.

```
num_vectors = 1000
dims = [2, 10, 100, 1000]
```

```
fig, axes = plt.subplots(1, 4, figsize=(18, 4), sharey=False)

for ax, d in zip(axes, dims):
    # 1. Generate random vectors (multivariate normal)
    vecs = np.random.normal(0, 1, (num_vectors, d))

    # 2. Normalize them to length 1 (project onto unit sphere)
    vecs /= np.linalg.norm(vecs, axis=1, keepdims=True)

    # 3. Calculate dot products (cosine of angle) using matrix multiplication
    # We only want the upper triangle (unique pairs, excluding self-angles)
    dot_products = vecs @ vecs.T
    upper_tri_indices = np.triu_indices(num_vectors, k=1)
    pair_dots = dot_products[upper_tri_indices]

    # 4. Convert to angles in degrees
    # (Clip to [-1, 1] to avoid floating point issues with arccos)
    angles = np.degrees(np.arccos(np.clip(pair_dots, -1.0, 1.0)))

    # 5. Plot
    ax.hist(angles, bins=60, color='teal', edgecolor='white', density=True)
    ax.axvline(90, color='red', linestyle='dashed', linewidth=2)
    ax.set_title(f'Dimension = {d}')
    ax.set_xlim(0, 180)
    ax.set_xlabel("Angle (degrees)")

axes[0].set_ylabel("Frequency")
plt.suptitle("Angles Between Random Vectors", fontsize=16, y=1.05)
plt.show()
```

The Takeaway:
In 2D, angles are uniformly distributed from 0° to 180°. But as dimensions increase, the histogram aggressively pinches inward.
By the time you reach 1000 dimensions, literally millions of random pairs of vectors all have an angle of precisely 90°. There is so much "room" at the equator of a high-dimensional sphere that every random point you pick is effectively the north pole to every other point's equator.

4. The Gaussian shell

People with an undergrad math background are intimately familiar with the bell curve. They naturally assume a multivariate Gaussian distribution N(0,Id) looks like a dense cloud of points clustered tightly around the origin, fading out at the edges.

In high dimensions, this intuition is completely wrong.

- **The Math:** The squared norm of a vector drawn from N(0,Id) follows a Chi-squared distribution with d degrees of freedom. The expected value of the squared distance from the origin is d, and the standard deviation grows only as sqrt(2d).
- **The Paradox:** This means virtually *zero* probability mass is located at the mean (the origin). Instead, almost all the points are concentrated in a incredibly thin "soap bubble" or annulus of radius ≈sqrt(d) around the origin.
- **Python Hook:** Generate `np.random.randn(N, d)`, calculate the Euclidean norm (distance from the origin) for each point, and plot the histograms. In 2D, it looks like a half-bell curve peaking at the origin. By d=100, it looks like a tight Gaussian perfectly centered miles away from the origin at r=10.